



NextGen

Efficient Python Lecture 06

CERN
STEAM Academy

13 July 2026

Measurement-Guided Optimization

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland



Outline

Measurement before optimization

Benchmarking and timing

Runtime profiling and call structure

Memory, debugging, and disciplined improvement

Further reading

Section 1

Measurement before optimization

From reliable programs to measured performance

In the previous lectures, we made numerical objects, data flow, state, and diagnostics more explicit. We now move to a new question: what should we do when a scientific Python program is too slow, too memory-hungry, or too difficult to trust?

data $\longrightarrow$ computation $\longrightarrow$ diagnostics $\longrightarrow$ measurement $\longrightarrow$ change

Performance work should not start from tricks. It should start from understanding what the program actually does.

A program that is difficult to read is usually difficult to measure. A program that is difficult to measure is dangerous to optimize.

The central principle

The central principle of this lecture is simple:

Do not optimize what you have not measured.

In scientific computing, this is not only a programming rule. It is an experimental rule. A performance claim should be supported by a measurement, and the measurement should have a clear experimental setup.

Before changing the code, we should ask:

- ▶ what quantity is being measured,
- ▶ what input is representative,
- ▶ what scale of the problem is relevant,
- ▶ whether the output is still scientifically correct,
- ▶ whether the observed improvement is reproducible.

What are we measuring?

The word “performance” is too vague. A slow program may be limited by very different effects.

- ▶ Runtime: how long does the computation take?
- ▶ Memory: how much memory is allocated, and what is the peak usage?
- ▶ I/O: how much time is spent reading, writing, parsing, or saving data?
- ▶ Algorithmic scaling: how does the cost grow when the problem becomes larger?
- ▶ Reliability: how quickly can we find and fix numerical or logical mistakes?

A useful measurement should tell us which of these questions we are answering. Otherwise, we may optimize the wrong part of the program.

Four complementary activities

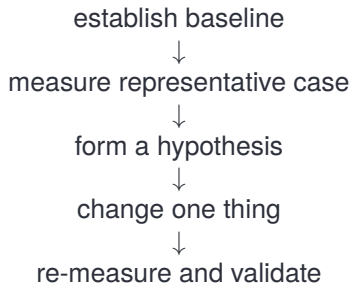
Timing, profiling, memory inspection, and debugging are related, but they answer different questions.

Activity	Question	Typical tool
Timing	How long does this take?	timeit, perf_counter
Profiling	Where is time spent?	cProfile, pstats
Memory inspection	Where is memory allocated?	tracemalloc, array diagnostics
Debugging	Why is the result wrong?	assertions, breakpoints, minimal examples

The tools support each other. Timing detects a problem. Profiling localizes it. Memory inspection explains allocation pressure. Debugging protects correctness before and after any change.

A disciplined optimization workflow

A useful optimization workflow is cyclic. Each step should be explicit enough that we can explain why the next step is justified.



The most important part is not the tool itself. The important part is the discipline: one hypothesis, one change, and a comparison with the original baseline.

First correctness, then speed

Fast wrong code is still wrong. Before optimizing, the computation should have at least minimal correctness checks.

```
1 import numpy as np
2
3 def energy_density(phi, dphi):
4     return 0.5*dphi**2 + 0.25*(phi**2 - 1.0)**2
5
6 x = np.linspace(-10.0, 10.0, 2001)
7 phi = np.tanh(x)
8 dphi = np.gradient(phi, x)
9
10 E = np.trapezoid(energy_density(phi, dphi), x)
11
12 assert np.isfinite(E)
13 assert E > 0.0
```

Assertions are not only for software engineers. In numerical work, they document assumptions about shapes, ranges, signs, units, and finite values.

Baselines as numerical references

A baseline is the reference version against which we compare future changes. It does not have to be fast. It has to be clear, correct, and reproducible.

```
1 def baseline_residual(u, dx):
2     """Residual of a simple 1D nonlinear boundary-value problem."""
3     r = np.zeros_like(u)
4
5     for i in range(1, len(u) - 1):
6         u_xx = (u[i-1] - 2*u[i] + u[i+1]) / dx**2
7         r[i] = -u_xx + u[i]*(u[i]**2 - 1.0)
8
9     return r
```

A good baseline is often simple Python. Later, we may replace it with vectorized, compiled, or parallel code. But the baseline remains useful for validation.

Representative inputs

A benchmark is only useful if the input resembles the real workload. A tiny toy input may measure overhead. A huge artificial input may measure memory pressure that is irrelevant to the actual project.

Representative inputs should reflect:

- ▶ the typical problem size,
- ▶ the typical data type,
- ▶ the typical array shape and memory layout,
- ▶ the typical number of repetitions or time steps,
- ▶ the typical I/O pattern, if files are involved.

For example, benchmarking a solver kernel on an array of length 100 may say little about a simulation that uses 10^6 grid points.

Input size is a physical parameter

In numerical physics, input size is rarely arbitrary. It often corresponds to a physical or numerical resolution.

$$N_x \uparrow \iff \Delta x \downarrow, \quad N_t \uparrow \iff \Delta t \downarrow \text{ or } t_{\max} \uparrow.$$

Changing the input size may change:

- ▶ the numerical accuracy,
- ▶ the stability restrictions,
- ▶ the amount of memory required,
- ▶ the balance between Python overhead and numerical work.

A scaling experiment is therefore both a performance test and a numerical experiment.

Runtime, memory, I/O, algorithm

Different bottlenecks require different changes. A program may be slow for reasons that have very different solutions.

Bottleneck	Possible symptom
Runtime	CPU time dominated by a repeated numerical kernel
Memory	large peak memory, swapping, allocation failures
I/O	most time spent loading, parsing, saving, or converting files
Algorithm	doubling input size increases runtime by a large factor
Python overhead	many small function calls or loops over scalar objects

The first measurement should help us decide which type of bottleneck we are dealing with.

Algorithmic scaling vs constant factors

Some optimizations reduce a constant factor. Others change the scaling law. These are not the same.

$$T(N) \approx CN^p.$$

Reducing C is useful. Reducing p can be transformative.

For small N , a slower-scaling algorithm may appear acceptable. For large N , the scaling exponent dominates. This is why timing one input size can be misleading.

In scientific computing, the most important optimization is sometimes not vectorization or compilation, but choosing a better numerical or algorithmic formulation.

A small physics-like workload

We will use a simple residual computation as a representative workload. It is small enough to understand, but close to many real numerical kernels.

```
1 import numpy as np
2
3 def make_problem(n):
4     x = np.linspace(-10.0, 10.0, n)
5     dx = x[1] - x[0]
6     u = np.tanh(x)
7     return x, u, dx
8
9 def residual(u, dx):
10    r = np.empty_like(u)
11    r[0] = 0.0
12    r[-1] = 0.0
13    r[1:-1] = (
14        -(u[:-2] - 2*u[1:-1] + u[2:])) / dx**2
15        + u[1:-1]*(u[1:-1]**2 - 1.0)
16    )
17    return r
```

Separate setup from measurement

A common benchmarking mistake is to time both the setup and the operation of interest. Sometimes this is intended. Often it is not.

```
1 # Measures grid construction and residual evaluation
2 x, u, dx = make_problem(100_000)
3 r = residual(u, dx)
4
5 # For kernel timing, prepare the input first
6 x, u, dx = make_problem(100_000)
7
8 # Timed part should be only this
9 r = residual(u, dx)
```

The setup may include allocation, random number generation, file reading, or preprocessing. Decide explicitly whether these operations belong to the benchmark.

Record the experimental context

A timing result without context is difficult to interpret. A useful benchmark should record not only the number, but also the conditions under which it was obtained.

- ▶ Python version and important library versions.
- ▶ Problem size and data type.
- ▶ Number of repetitions.
- ▶ Machine or cluster node type.
- ▶ Number of threads used by numerical libraries, if relevant.
- ▶ Whether the benchmark was run in a notebook, script, or batch job.

For teaching examples, we do not need perfect benchmarking infrastructure. But we should cultivate the habit that timings are experimental data.

Section 2

Benchmarking and timing

Timing is an experiment

Timing is the simplest form of performance measurement, but it is not trivial. Two runs of the same code may differ because the computer is not an isolated experimental system.

Possible sources of variation include:

- ▶ operating-system scheduling,
- ▶ CPU frequency changes,
- ▶ memory allocation and garbage collection,
- ▶ cache effects,
- ▶ background processes,
- ▶ initialization costs in libraries.

Therefore, a single timing number should be treated with suspicion. Repetition and comparison are essential.

A naive timing attempt

A very common first attempt is to use the current time before and after a piece of code.

```
1 import time
2
3 start = time.time()
4
5 x, u, dx = make_problem(100_000)
6 r = residual(u, dx)
7
8 end = time.time()
9 print(end - start)
```

This may be enough for a rough impression, but it is usually not enough for a careful comparison. It mixes setup with computation, gives only one sample, and uses a timer that is not the best choice for benchmarking.

Coarse timing with `perf_counter`

For measuring elapsed wall-clock time, `time.perf_counter` is the usual choice. It provides a high-resolution monotonic timer.

```
1 from time import perf_counter
2
3 x, u, dx = make_problem(100_000)
4
5 start = perf_counter()
6 r = residual(u, dx)
7 elapsed = perf_counter() - start
8
9 print(f"elapsed time: {elapsed:.6f} s")
```

This is useful for coarse measurements: one solver run, one data-processing step, one simulation, or one complete analysis pipeline.

Repeating measurements

A single measurement is fragile. A small helper function can already give a better picture.

```
1 from time import perf_counter
2 import numpy as np
3
4 def measure_once(func):
5     start = perf_counter()
6     result = func()
7     elapsed = perf_counter() - start
8     return elapsed, result
9
10 times = []
11 for _ in range(10):
12     elapsed, r = measure_once(lambda: residual(u, dx))
13     times.append(elapsed)
14
15 times = np.array(times)
16
17 print(times.min(), np.median(times), times.max())
```

The spread of the values is part of the information.

timeit for small statements

For small pieces of code, `timeit` is more convenient than manually writing timing loops. It repeats the statement many times and reports repeated measurements.

```
1 import timeit
2 import numpy as np
3
4 x, u, dx = make_problem(100_000)
5
6 samples = timeit.repeat(
7     "residual(u, dx)",
8     globals={"residual": residual, "u": u, "dx": dx},
9     number=100,
10    repeat=7,
11 )
12
13 per_call = np.array(samples) / 100
14
15 print("best:", per_call.min())
16 print("median:", np.median(per_call))
```

Using `timeit` with functions

For anything more than a tiny expression, it is usually better to benchmark a function call rather than a loose fragment of code.

```
1 def compute_residual_for_benchmark():
2     return residual(u, dx)
3
4 samples = timeit.repeat(
5     "compute_residual_for_benchmark()",
6     globals={
7         "compute_residual_for_benchmark":
8             compute_residual_for_benchmark
9     },
10    number=100,
11    repeat=7,
12 )
```

This makes the timed operation easier to name, reuse, validate, and replace by another implementation.

Warm-up and initialization effects

The first execution of a piece of code may not be representative. It may include imports, memory allocation, cache effects, or library initialization.

```
1 x, u, dx = make_problem(100_000)
2
3 # Warm-up: run once before collecting timings
4 r = residual(u, dx)
5
6 samples = timeit.repeat(
7     "residual(u, dx)",
8     globals={"residual": residual, "u": u, "dx": dx},
9     number=100,
10    repeat=7,
11 )
```

Warm-up is especially important when later using just-in-time compilation, GPUs, large libraries, or workflows that allocate large temporary arrays.

Reproducible inputs for timing

If the input is random, generate it reproducibly and outside the timed statement, unless input generation is part of what you want to measure.

```
1 def make_random_field(n, seed=123):
2     rng = np.random.default_rng(seed)
3     return rng.normal(size=n)
4
5 u = make_random_field(100_000, seed=123)
6 dx = 1.0 / (u.size - 1)
7
8 # The benchmark measures the residual computation,
9 # not random number generation.
10 samples = timeit.repeat(
11     "residual(u, dx)",
12     globals={"residual": residual, "u": u, "dx": dx},
13     number=100,
14     repeat=7,
15 )
```

Comparing two implementations

When comparing two versions, use the same input and check that the numerical results agree.

```
1 def residual_reference(u, dx):
2     return baseline_residual(u, dx)
3 def residual_candidate(u, dx):
4     return residual(u, dx)
5 r_ref = residual_reference(u, dx)
6 r_new = residual_candidate(u, dx)
7 assert np.allclose(r_ref, r_new)
8 t_ref = timeit.repeat(
9     "residual_reference(u, dx)",
10    globals=globals(), number=20, repeat=5
11 )
12 t_new = timeit.repeat(
13     "residual_candidate(u, dx)",
14    globals=globals(), number=20, repeat=5
15 )
```

Performance comparisons without correctness checks can be very misleading.

Interpreting variance

Timing results are samples, not exact constants. A useful summary should show both a typical value and the spread.

```
1 import numpy as np
2
3 t = np.array(t_new) / 20
4
5 print(f"min      = {t.min():.6e} s")
6 print(f"median  = {np.median(t):.6e} s")
7 print(f"std     = {t.std(ddof=1):.6e} s")
```

The minimum often estimates a best-case lower bound with less external interference. The median is often a more realistic typical value. A large spread means that the experiment is noisy and should not be over-interpreted.

Timing across sizes: the first decision point

A benchmark at one size answers only one question. A scaling experiment is often more informative.

```
1 sizes = [1_000, 3_000, 10_000, 30_000, 100_000]
2 best_times = []
3
4 for n in sizes:
5     x, u, dx = make_problem(n)
6     samples = timeit.repeat(
7         "residual(u, dx)",
8         globals={"residual": residual, "u": u, "dx": dx},
9         number=50,
10        repeat=5,)
11     best_times.append(min(samples) / 50)
12
13 for n, t in zip(sizes, best_times):
14     print(f"{n:8d}  {t:.6e} s")
```

At this point we can decide what is justified: a better algorithm, vectorization, profiling, memory inspection, or simply no optimization at all.

→ **Example 06 - 01**

Section 3

Runtime profiling and call structure

From timing to profiling

Timing answers the question:

How long does this computation take?

Profiling asks a different question:

Where is the time spent?

This distinction is essential. A benchmark may tell us that a simulation takes 20 seconds, but it does not tell us whether the expensive part is the solver kernel, interpolation, plotting, file output, repeated allocation, or a small function called millions of times.

Profiling is the tool we use when the total time is not enough information to justify a code change.

A workload with a call structure

Profiling becomes useful when a program has a nontrivial call structure. For example, a time-stepping computation may call a residual function many times.

```
1 def step(u, dx, dt):
2     r = residual(u, dx)
3     return u - dt*r
4
5 def run_simulation(n=100_000, n_steps=200):
6     x, u, dx = make_problem(n)
7
8     for _ in range(n_steps):
9         u = step(u, dx, dt=1e-4)
10
11     return u
```

The total runtime is produced by many nested operations. Profiling helps us decide which level deserves attention.

Profiling a whole script

A simple way to profile a complete script is to run it through `cProfile` from the command line.

```
1 python -m cProfile -s cumtime simulate.py
```

The option `-s cumtime` sorts the output by cumulative time. This is often a good first view because it shows which high-level functions dominate the complete execution. The profile can also be saved to a file:

```
1 python -m cProfile -o profile.stats simulate.py
```

Saving the profile is useful when the run is expensive or when we want to inspect the result later.

Profiling from Python

For notebooks, teaching examples, and selected functions, it is convenient to profile directly from Python.

```
1 import cProfile
2 import pstats
3 import io
4 from pstats import SortKey
5
6 profiler = cProfile.Profile()
7 profiler.enable()
8 u_final = run_simulation(n=100_000, n_steps=200)
9 profiler.disable()
10 stream = io.StringIO()
11 stats = pstats.Stats(profiler, stream=stream)
12 stats.strip_dirs()
13 stats.sort_stats(SortKey.CUMULATIVE)
14 stats.print_stats(12)
15 print(stream.getvalue())
```

This gives a compact report of the most important functions.

Reading the profiler table

A typical `cProfile` table contains columns like these:

Column	Meaning
<code>ncalls</code>	number of calls to the function
<code>totttime</code>	time spent inside the function itself, excluding subcalls
<code>percall</code>	time per call
<code>cumtime</code>	time spent in the function and all functions called by it
<code>filename:lineno(function)</code>	location and name of the function

The table is not just a list of slow functions. It is a compressed description of the execution structure of the program.

Internal time and cumulative time

The difference between `tottime` and `cumtime` is one of the most important parts of profiler output.

`tottime` = time spent directly in the function,

`cumtime` = time spent in the function and its subcalls.

A function with high `cumtime` may be an important high-level driver. A function with high `tottime` may contain the local operation that actually consumes CPU time.

For scientific code, high cumulative time often points to the stage of the workflow. High internal time often points to the kernel.

Call counts matter

A function does not have to be slow once to become expensive. It may be cheap, but called many times.

```
1 def local_force(u):
2     return u*(u**2 - 1.0)
3
4 def residual_loop(u, dx):
5     r = np.zeros_like(u)
6
7     for i in range(1, len(u) - 1):
8         u_xx = (u[i-1] - 2*u[i] + u[i+1]) / dx**2
9         r[i] = -u_xx + local_force(u[i])
10
11     return r
```

Here, `local_force` may be called once for every interior grid point. The individual call is small, but the call count can be enormous.

Sorting and filtering statistics

The same profile can be viewed in different ways. Sorting by cumulative time and sorting by internal time answer different questions.

```
1 stats = pstats.Stats(profiler)
2
3 # Which high-level functions dominate the run?
4 stats.strip_dirs()
5 stats.sort_stats(SortKey.CUMULATIVE)
6 stats.print_stats(10)
7
8 # Which functions spend most time internally?
9 stats.sort_stats(SortKey.TIME)
10 stats.print_stats(10)
```

The profiler does not automatically tell us what to rewrite. It gives us evidence for a hypothesis.

Caller and callee views

Sometimes we need to know not only which function is expensive, but why it was called.

```
1 stats = pstats.Stats(profiler)
2 stats.strip_dirs()
3 stats.sort_stats(SortKey.CUMULATIVE)
4
5 # Who called the expensive functions?
6 stats.print_callers(10)
7
8 # What did the expensive functions call?
9 stats.print_callees(10)
```

This is useful when the same function is used in several contexts. The problem may not be the function itself, but the place where it is called too often.

Profile representative runs

A profile should be representative, but it does not have to be the largest possible run. For example, instead of profiling a full production simulation, we may use:

- ▶ the same code path,
- ▶ the same array shapes or similar scaling regime,
- ▶ fewer time steps,
- ▶ smaller but not tiny grids,
- ▶ the same file formats and preprocessing steps, if I/O matters.

The goal is to reproduce the bottleneck without waiting unnecessarily long. A profile of a toy case may be misleading; a profile of a full production run may be inconvenient.

A profiler is not a benchmark

Profiling changes the execution. It introduces overhead because the program is being observed.

profiled time $\neq$ unprofiled benchmark time.

Therefore:

- ▶ use `timeit` or `perf_counter` for timing comparisons,
- ▶ use `cProfile` to localize where time is spent,
- ▶ do not compare optimized and non-optimized versions only from profiler totals,
- ▶ after changing the code, benchmark again without profiling.

Profiling is diagnostic. Benchmarking is comparative.

When profiler output surprises you

Profiler output is often surprising. That is precisely why we use it.

Possible surprises:

- ▶ the expensive part is not the mathematical kernel,
- ▶ plotting or saving dominates the runtime,
- ▶ many tiny Python calls dominate the execution,
- ▶ a supposedly vectorized function is called inside a loop,
- ▶ setup is accidentally repeated many times,
- ▶ the bottleneck appears only for larger inputs.

A useful reaction is not immediately to rewrite the code. First ask what hypothesis explains the measurement.

From profile to decision

After profiling, we should decide what kind of change is justified.

Observation	Possible response
One numerical kernel dominates	optimize or replace that kernel
Many scalar Python calls dominate	reduce Python-level loops or function calls
I/O dominates	change file format, reduce frequency, buffer output
High-level driver dominates through subcalls	inspect algorithm or repeated workflow stages
No clear hotspot	problem may be memory, I/O, scaling, or measurement noise

The profile should narrow the space of possible actions.

→ **Example 06 - 02**

Section 4

Memory, debugging, and disciplined improvement

Memory is a different bottleneck

Runtime profiling is not enough when the program is limited by memory.

A memory problem may appear as:

- ▶ unexpectedly large peak memory,
- ▶ slow execution caused by memory pressure,
- ▶ temporary arrays created inside vectorized expressions,
- ▶ accidental copies of large arrays,
- ▶ failure only for larger grids or larger batches.

In numerical Python, memory and runtime are strongly connected. A code change that reduces arithmetic may still be slower if it creates large temporary arrays.

Estimate memory before running

Before running a large computation, estimate the size of the main arrays.

```
1 import numpy as np
2
3 def array_size_MB(shape, dtype=np.float64):
4     n_values = np.prod(shape)
5     itemsize = np.dtype(dtype).itemsize
6     return n_values * itemsize / 1024**2
7
8 print(array_size_MB((2000, 2000)))      # float64 field
9 print(array_size_MB((100, 2000, 2000))) # 100 snapshots
```

For float64, one number uses 8 bytes. A single 2000×2000 field is already about 30.5 MB. A time series of many such fields may quickly become the main limitation.

Inspect actual array sizes

NumPy arrays know how much memory their data buffer uses.

```
1 x = np.linspace(-10.0, 10.0, 2_000_000)
2 u = np.tanh(x)
3
4 print(u.shape)
5 print(u.dtype)
6 print(u.nbytes / 1024**2, "MB")
```

This does not describe the full memory footprint of a complex program, but it is an excellent first check. In scientific code, large memory use is often already visible from the main arrays.

Peak memory can exceed final memory

The final array is not the whole story. Intermediate arrays may temporarily increase memory use.

```
1 n = 5_000_000
2
3 a = np.ones(n)
4 b = np.ones(n)
5 c = np.ones(n)
6
7 y = a*b + c
```

The final result `y` is one array. But the expression may also create a temporary array for `a*b` before adding `c`. For large arrays, temporary objects can matter as much as the final result.

Temporary arrays in vectorized code

Vectorization reduces Python overhead, but it does not automatically minimize memory traffic.

$$y = \sin(u)^2 + \cos(u)^2$$

may look like one mathematical expression, but computationally it may involve several array-sized temporaries.

```
1 y = np.sin(u)**2 + np.cos(u)**2
```

For small and medium arrays, this is perfectly fine. For very large arrays, memory allocation and memory bandwidth may become the limiting factors.

Reducing temporaries carefully

Some NumPy operations support an `out` argument. This allows us to reuse an existing array.

```
1 tmp = np.empty_like(u)
2 y = np.empty_like(u)
3
4 np.sin(u, out=tmp)
5 np.square(tmp, out=y)
6
7 np.cos(u, out=tmp)
8 np.square(tmp, out=tmp)
9
10 y += tmp
```

This version is longer and less mathematical, but it can reduce temporary allocations. Such code should be used only when memory measurements justify it, because readability is also valuable.

Views and copies revisited

Memory inspection is connected to concepts from idiomatic scientific Python: identity, mutation, views, and copies.

```
1 u = np.arange(10.0)
2
3 v = u[2:8]          # usually a view
4 w = u[2:8].copy()
5
6 print(np.shares_memory(u, v)) # True
7 print(np.shares_memory(u, w)) # False
```

A view is memory-efficient, but it is not independent. A copy is safer when independent modification is needed, but it uses additional memory.

The right choice depends on the intended data flow.

Tracing current and peak memory

The `tracemalloc` module can trace Python memory allocations and report current and peak traced memory.

```
1 import tracemalloc
2
3 tracemalloc.start()
4
5 u_final = run_simulation(n=100_000, n_steps=200)
6
7 current, peak = tracemalloc.get_traced_memory()
8 tracemalloc.stop()
9
10 print(f"current: {current / 1024**2:.2f} MB")
11 print(f"peak:    {peak / 1024**2:.2f} MB")
```

Peak memory is often more important than final memory. A program may finish with moderate memory use, but still fail for larger inputs because the peak is too high.

Snapshot comparison with tracemalloc

Snapshots allow us to compare memory allocations before and after a computation.

```
1 import tracemalloc
2
3 tracemalloc.start()
4
5 snapshot_before = tracemalloc.take_snapshot()
6
7 u_final = run_simulation(n=100_000, n_steps=200)
8
9 snapshot_after = tracemalloc.take_snapshot()
10
11 stats = snapshot_after.compare_to(
12     snapshot_before,
13     key_type="lineno",)
14
15 for stat in stats[:10]:
16     print(stat)
17
18 tracemalloc.stop()
```

This helps locate lines that are responsible for increased traced memory allocation.

Interpreting memory traces

Memory tools also require interpretation.

- ▶ A line with large allocation is not automatically wrong.
- ▶ Allocation may be necessary, but repeated allocation may be avoidable.
- ▶ Peak memory may be caused by temporaries that disappear quickly.
- ▶ Python-level tracing is not always the same as total resident memory reported by the operating system.
- ▶ Array sizes, `nbytes`, profiler output, and system monitoring should be interpreted together.

For large numerical programs, memory reasoning is often a combination of estimates, tracing, and knowledge of the numerical method.

→ **Example 06 - 03**

Debugging is also part of optimization

Debugging may look separate from performance, but in practice it is part of the same disciplined workflow.

Before optimizing, we need to know:

- ▶ whether the current result is correct,
- ▶ which assumptions the code relies on,
- ▶ where the computation first becomes invalid,
- ▶ whether an optimization has changed the answer.

A performance improvement that silently changes the scientific result is not an improvement. Debugging protects the meaning of the measurement.

Assertions as numerical contracts

Assertions are simple, but very useful. They make assumptions executable.

```
1 def residual_checked(u, dx):
2     assert u.ndim == 1
3     assert u.size >= 3
4     assert dx > 0.0
5     assert np.isfinite(u).all()
6
7     r = residual(u, dx)
8
9     assert r.shape == u.shape
10    assert np.isfinite(r).all()
11
12    return r
```

Assertions are especially useful at interfaces: after loading data, before entering a solver, after reshaping, after computing diagnostics, and before saving results.

Minimal failing examples

When something is wrong, reduce the problem until the failure becomes visible.

```
1 # Instead of debugging the full simulation:
2 u_final = run_simulation(n=1_000_000, n_steps=10_000)
3
4 # Try to isolate the failing operation:
5 x, u, dx = make_problem(20)
6
7 r1 = baseline_residual(u, dx)
8 r2 = residual(u, dx)
9
10 print(r1)
11 print(r2)
12 print(np.max(np.abs(r1 - r2)))
```

A minimal example should be small enough to inspect by eye, but still preserve the essential structure of the bug.

Breakpoints

Print statements are useful, but sometimes we need to stop the program and inspect the local state interactively.

```
1 def step_checked(u, dx, dt):
2     r = residual_checked(u, dx)
3
4     if not np.isfinite(r).all():
5         breakpoint()
6
7     return u - dt*r
```

At the breakpoint, we can inspect variables such as `u.shape`, `u.min()`, `u.max()`, `dx`, or individual slices of the array.

A breakpoint is most useful when placed near the first suspicious transition, not randomly at the end of the program.

Debugging a numerical loop

In simulations and training loops, bugs often appear only after many iterations. It is useful to monitor compact diagnostics.

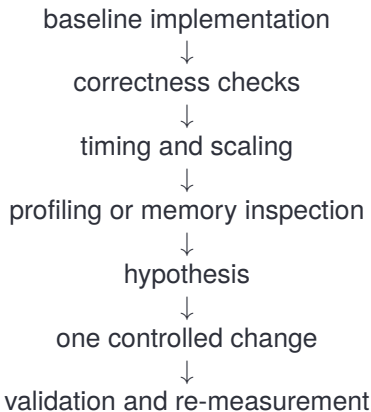
```
1 def run_checked(n=100_000, n_steps=200):
2     x, u, dx = make_problem(n)
3
4     for k in range(n_steps):
5         u = step_checked(u, dx, dt=1e-10)
6
7         if k % 20 == 0:
8             norm = np.linalg.norm(u)
9             maximum = np.max(np.abs(u))
10
11             assert np.isfinite(norm)
12             assert np.isfinite(maximum)
13
14             print(k, norm, maximum)
15     return u
```

Diagnostics should be cheap, meaningful, and connected to the physics or numerical method.

→ **Example 06 - 04**

The full improvement cycle

The complete workflow is a cycle, not a sequence of tricks.



The phrase “one controlled change” is important. If we change the algorithm, data layout, dtype, and solver tolerance at the same time, we no longer know what caused the result.

What this lecture prepares us for

After measurement, we can make informed optimization decisions.

Possible next steps include:

- ▶ changing the algorithm,
- ▶ reducing Python-level loops,
- ▶ improving data layout,
- ▶ reducing temporary arrays,
- ▶ using compiled kernels,
- ▶ parallelizing independent work,
- ▶ moving suitable workloads to accelerators.

These are the topics of acceleration and scalable execution. The key point is that acceleration should come after measurement, not before it.

Section 5

Further reading

Supplementary literature

- ▶ Crapé, A., Eeckhout, L. A rigorous benchmarking and performance analysis methodology for Python workloads. In: *2020 IEEE International Symposium on Workload Characterization (IISWC)*, 83–93 (2020). DOI: <https://doi.org/10.1109/IISWC50251.2020.00017>
- ▶ Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F. Producing wrong data without doing anything obviously wrong! In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, 265–276 (2009). DOI: <https://doi.org/10.1145/1508244.1508275>
- ▶ Fleming, P.J., Wallace, J.J. How not to lie with statistics: the correct way to summarize benchmark results. *Communications of the ACM* **29**(3), 218–221 (1986). DOI: <https://doi.org/10.1145/5666.5673>
- ▶ Berger, E.D., Stern, S., Altmayer Pizzorno, J. Triangulating Python performance issues with SCALENE. In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 51–64 (2023). URL: <https://www.usenix.org/conference/osdi23/presentation/berger>